

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: OPERATING SYSTEMS COURSE CODE: CSA04

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance (BL3-BL4)

Assessment Tool Weightage: 30%

QUESTIONS: 10

Total Marks:50

Annexure A

Scenario based

Code of Conduct:

I R. Indhu (192524332) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

R. Indhu
Signature of the student:

Question 1 — CPU Scheduling in a Real-Time Ticket Booking System

A railway ticket booking system experiences heavy load during festival seasons. The system handles three categories of processes:

- P1 – Payment gateway transactions (High priority)
- P2 – Seat availability checks (Medium priority)
- P3 – Ticket confirmation and printing (Low priority)
- Burst times vary from 2 ms to 20 ms.

Recently, customers complained that ticket confirmations are taking too long.

Tasks

1. Identify the most suitable scheduling algorithm (FCFS, SJF, Priority, RR) for this scenario.
2. Justify *why* your chosen method improves CPU utilization and reduces customer wait time.
3. Provide a sample execution order for the above categories.
4. Discuss one drawback of this scheduling method in real-time systems.

(10 Marks)

Question 2 — Deadlock in a Banking Transaction Server

A bank server handles the following operations simultaneously:

- T1: Transfers money between accounts
- T2: Updates passbook entries
- T3: Generates monthly statements

Due to simultaneous locking of Account Database (A), Customer Info Database (B), and Transaction Logs (C), the system enters a state where:

- T1 holds A and requests B
- T2 holds B and requests C
- T3 holds C and requests A

Tasks

1. Draw the resource request situation (text-based RAG).
2. Identify whether a deadlock has occurred and justify your answer.

3. Recommend *one specific technique* (avoidance/prevention/recovery) and explain how it fixes the issue.
4. Suggest a redesign for the locking order to avoid future deadlocks.

(10 Marks)

Question 3 — Mobile OS App Switching Lag

A mobile operating system is designed to support smooth app switching.

But users report that:

- Opening WhatsApp delays the background music player.
- Switching between social media apps feels laggy.
- CPU shows high idle times despite multiple running apps.

The OS currently uses Round Robin (quantum = 20 ms) with 20 background apps active.

Tasks

1. Analyze why the CPU remains idle despite many active processes.
2. Determine whether the time quantum is helping or hurting performance.
3. Recommend a better scheduling method or quantum value, with justification.
4. Explain how the change will reduce switching lag and improve system throughput.

(10 Marks)

Question 4 — Deadlock Risk in an Operating System Update

During a Windows OS update, several modules run in parallel:

- M1: Copy update files
- M2: Install drivers
- M3: Update registry
- M4: Validate system integrity

Resources involved:

- R1 = Disk I/O
- R2 = Kernel lock
- R3 = Configuration lock

Recently logs show:

- M1 holds R1, waiting for R2

- M2 holds R2, waiting for R3
- M3 holds R3, waiting for R1

Tasks

1. Identify whether this sequence can lead to a deadlock.
2. If yes, explain the exact circular wait condition.
3. Propose *Banker's algorithm* style resource allocation to avoid risk.
4. Suggest how OS designers can prioritize modules to improve CPU utilization during update.

(10 Marks)

Question 5 — Cloud Server CPU Utilization Analysis

A cloud hosting provider notices:

- 80% of the time, VM processes wait for network or disk
- Only 20% of the time is spent on CPU
- Each server runs 7 VMs

Using the formula:

$$U = 1 - p^n$$

where p = I/O wait probability

Tasks

1. Calculate CPU utilization.
2. Interpret whether the server is underutilized or optimally loaded.
3. Suggest how increasing or decreasing number of VMs affects throughput.
4. Propose two OS-level strategies for improving CPU utilization in this cloud environment.

(10 Marks)

Question 6 — Hospital Emergency System Process Scheduling

A hospital information system manages:

- P1 – Emergency patient registration (Highest Priority)
- P2 – Laboratory report generation (Medium Priority)
- P3 – Pharmacy billing (Low Priority)

During peak hours, pharmacy billing is delayed while emergency requests continue to arrive.

Tasks

1. Identify the scheduling issue occurring in the system.
2. Recommend a suitable CPU scheduling algorithm.
3. Explain how starvation can be prevented.
4. Discuss how your solution improves response time and fairness.

(10 Marks)

Question 7 — E-Commerce Warehouse Synchronization

An e-commerce warehouse has multiple worker threads updating a shared inventory database simultaneously.

Developers observe:

- Incorrect stock counts
- Duplicate order confirmations
- Frequent data inconsistencies

Tasks

1. Identify the synchronization problem.
2. Explain why a critical section is required.
3. Recommend either semaphores or monitors to solve the issue.
4. Explain how the proposed solution maintains data consistency.

(10 Marks)

Question 8 — Video Streaming Service Thread Management

A video streaming platform creates one thread for every client connection.

During a live sports event:

- 60,000 users connect simultaneously.
- CPU utilisation reaches 95%.
- Memory consumption increases rapidly.
- Video buffering becomes frequent.

Tasks

1. Identify the thread-management problem.
2. Recommend a suitable multithreading model.
3. Suggest two optimisations to improve scalability.
4. Explain how your recommendations improve throughput.

(10 Marks)

Question 9 — Smart Traffic Control Deadlock

A smart city traffic management system controls four intersections.

Resources:

- R1 – North Junction
- R2 – East Junction
- R3 – South Junction
- R4 – West Junction

Current situation:

- T1 holds R1 and requests R2.
- T2 holds R2 and requests R3.
- T3 holds R3 and requests R4.
- T4 holds R4 and requests R1.

Tasks

1. Determine whether a deadlock exists.
2. Explain the four necessary conditions for deadlock in this scenario.
3. Recommend one recovery technique.
4. Suggest a redesign that prevents similar deadlocks.

(10 Marks)

Question 10 — University Examination Server Resource Allocation

An online examination server allocates CPU, memory and database connections to student sessions.

At examination time:

- Thousands of students log in simultaneously.
- New resource requests arrive continuously.
- The administrator wants to avoid deadlocks without reducing resource utilisation.

Tasks

1. Explain how Banker's Algorithm can be applied.
2. Describe the information required before granting a request.
3. Explain how the system determines whether it is in a safe state.
4. Discuss one limitation of using Banker's Algorithm in large-scale cloud systems.

(10 Marks)

RUBRICS FOR CALCULATION

Scenario based assessment

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding ; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Marks Evaluation:

Question No.	Understanding of Scenario (20%)	Identification of Key Issues (20%)	Application of Concepts (25%)	Decision Making (20%)	Clarity of Response (15%)	Total Marks(100)
Q1						
Q2						
Q3						
Q4						
Q5						
Q6						

Q7						
Q8						
Q9						
Q10						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Question 1 – CPU Scheduling in a Real-Time Ticket Booking System

1. Suitable Scheduling Algorithm:

The most suitable scheduling algorithm is Priority Scheduling because payment transactions are the most important and must be processed before other tasks.

2. Justification:

- High-priority payment transactions are executed immediately.
- Waiting time for critical processes is reduced.
- CPU utilization improves because important tasks are completed without unnecessary delay.
- Customers receive faster payment confirmation and better service.

3. Sample Execution Order:

P1 (Payment Gateway) → P2 (Seat Availability Check) → P3 (Ticket Confirmation & Printing)

4. Drawback:

Priority Scheduling may cause starvation, where low-priority processes such as ticket confirmation wait for a long time if high-priority processes continue to arrive.

Question 2 – Deadlock in a Banking Transaction Server

1. Text-Based Resource Allocation Graph (RAG):

T1 holds A → requests B

T2 holds B → requests C

T3 holds C → requests A

This creates the cycle:

A → T1 → B → T2 → C → T3 → A

2. Deadlock Identification:

Yes, a deadlock has occurred because each transaction is waiting for a resource held by another transaction, forming a circular wait.

3. Recommended Technique:

Deadlock Prevention is recommended.

The system should enforce a fixed order of resource allocation such as:

$A \rightarrow B \rightarrow C$

Since every transaction follows the same order, circular waiting is eliminated.

4. Locking Order Redesign:

All transactions should request resources only in the order:

Account Database (A) $\rightarrow$ Customer Information Database (B) $\rightarrow$ Transaction Logs (C)

This prevents circular wait and avoids future deadlocks.

Question 3 – Mobile OS App Switching Lag

1. Why does the CPU remain idle?

The CPU remains idle because many applications are waiting for I/O operations such as network communication or storage access. During these waiting periods, the CPU has no ready process to execute, resulting in idle time.

2. Is the 20 ms time quantum helping or hurting performance?

The 20 ms time quantum hurts performance for interactive mobile applications. A large time quantum increases response time, making app switching slower and causing noticeable lag.

3. Recommendation

Use Round Robin Scheduling with a smaller time quantum (10 ms) or Multilevel Feedback Queue (MLFQ) scheduling.

Justification:

- Faster response for interactive applications.
- Better CPU utilization.
- Improved user experience during app switching.

4. How will this improve performance?

- Reduces app switching delay.
- Improves response time.
- Increases CPU utilization.
- Improves overall system throughput.

- Provides smoother multitasking.

Question 4 – Deadlock Risk in an Operating System Update

1. Can this sequence lead to a deadlock?

Yes. This sequence can lead to a deadlock because each module is waiting for a resource held by another module.

2. Explain the circular wait condition.

The circular wait is:

- M1 holds R1 and waits for R2.
- M2 holds R2 and waits for R3.
- M3 holds R3 and waits for R1.

This forms the cycle:

M1 → R2 → M2 → R3 → M3 → R1 → M1

Since each module is waiting indefinitely for another, a deadlock occurs.

3. Banker's Algorithm Resource Allocation

Before allocating any requested resource, the operating system should check whether granting the request keeps the system in a safe state.

- If the system remains safe, allocate the resource.
- If not, postpone the request until resources become available.

This prevents deadlock from occurring.

4. Module Prioritization

The operating system should assign higher priority to critical update modules such as copying update files and installing drivers.

Benefits include:

- Faster completion of important tasks.
- Better CPU utilization.
- Reduced waiting time.
- More efficient completion of the update process.

Question 5 – Cloud Server CPU Utilization Analysis

1. Calculate CPU Utilization

Given:

- I/O wait probability (p) = 0.8
- Number of VMs (n) = 7

Formula:

$$U = 1 - p^n$$

$$U = 1 - (0.8)^7$$

$$U = 1 - 0.2097$$

$$U = 0.7903$$

$$\text{CPU Utilization} = 79.03\%$$

2. Interpretation

The CPU utilization is approximately 79.03%, which indicates that the server is optimally utilized. Most of the CPU resources are being used effectively while maintaining good system performance.

3. Effect of Increasing or Decreasing the Number of VMs

- Increasing the number of VMs can improve throughput because the CPU can execute other VMs while some are waiting for I/O operations. However, too many VMs may increase context switching and reduce performance.
- Decreasing the number of VMs reduces context-switching overhead but may leave the CPU idle more often, decreasing overall throughput.

4. Two OS-Level Strategies to Improve CPU Utilization

- Use efficient CPU scheduling algorithms to minimize CPU idle time.
- Implement load balancing to distribute workloads evenly across available servers and processors.

Question 6 – Hospital Emergency System Process Scheduling

1. Scheduling Issue

The scheduling issue is starvation. Pharmacy billing (low-priority process) waits for a long time because emergency registration requests (high-priority processes)

continuously arrive.

2. Suitable CPU Scheduling Algorithm

The most suitable scheduling algorithm is Priority Scheduling with Aging.

This algorithm allows emergency processes to receive immediate attention while gradually increasing the priority of waiting lower-priority processes.

3. Prevention of Starvation

Starvation can be prevented using the Aging technique.

In Aging, the priority of a process increases gradually as its waiting time increases. Eventually, even low-priority processes receive CPU time and are executed.

4. Improvement in Response Time and Fairness

Using Priority Scheduling with Aging provides the following benefits:

- Emergency patients receive immediate service.
- Pharmacy billing is not delayed indefinitely.
- Response time for critical tasks is improved.
- Fair CPU allocation is ensured for all processes.
- Overall system performance and customer service are enhanced.

Question 7 – E-Commerce Warehouse Synchronization

1. Identify the Synchronization Problem

The synchronization problem is a Race Condition. Multiple worker threads access and modify the shared inventory database simultaneously, leading to incorrect stock counts and duplicate order confirmations.

2. Why is a Critical Section Required?

A critical section is required because the shared inventory database is a common resource. If multiple threads update it at the same time, data may become inconsistent.

The critical section ensures that only one thread can access and update the shared database at a time, preventing conflicts.

3. Recommended Solution

The recommended synchronization mechanism is a Semaphore (Binary Semaphore/Mutex).

A binary semaphore allows only one thread to enter the critical section while other threads wait until the resource becomes available.

4. How Does the Solution Maintain Data Consistency?

Using a semaphore ensures:

- Only one thread updates the inventory database at a time.
- Prevents race conditions.
- Eliminates duplicate order confirmations.
- Maintains correct stock counts.
- Ensures data consistency and reliable transaction processing.

Question 8 – Video Streaming Service Thread Management

1. Identify the Thread-Management Problem

The problem is excessive thread creation. Creating one thread for each client connection consumes a large amount of CPU time and memory, reducing system performance.

2. Suitable Multithreading Model

The most suitable model is the Thread Pool Model.

In this model, a fixed number of threads are created in advance and reused to handle multiple client requests, reducing the overhead of creating and destroying threads.

3. Two Optimizations to Improve Scalability

Optimization 1: Use a Thread Pool to reuse existing threads instead of creating a new thread for every client.

Optimization 2: Use Asynchronous (Non-blocking) I/O so that threads do not remain idle while waiting for network or disk operations.

4. How Do These Recommendations Improve Throughput?

Using a thread pool and asynchronous I/O provides the following benefits:

- Reduces CPU overhead due to thread creation.
- Lowers memory consumption.
- Handles a large number of client requests efficiently.

- Reduces video buffering.
- Improves scalability and overall system throughput.

Question 9 – Smart Traffic Control Deadlock

1. Determine Whether a Deadlock Exists

Yes, a deadlock exists. Each traffic control process is waiting for a resource held by another process, creating a circular chain in which no process can proceed.

2. Explain the Four Necessary Conditions for Deadlock

The four necessary conditions are:

a) Mutual Exclusion:

Each junction resource (R1, R2, R3, R4) can be used by only one process at a time.

b) Hold and Wait:

Each process holds one resource while waiting to obtain another resource.

c) No Preemption:

Resources cannot be taken away from a process until it voluntarily releases them.

d) Circular Wait:

The waiting sequence is:

$T1 \rightarrow T2 \rightarrow T3 \rightarrow T4 \rightarrow T1$

forming a circular chain, which causes deadlock.

3. Recommended Recovery Technique

A suitable recovery technique is Resource Preemption.

The operating system can temporarily take a resource from one process or terminate one blocked process to break the circular wait. Once the cycle is broken, the remaining processes continue their execution normally.

4. Redesign to Prevent Similar Deadlocks

Assign a fixed order for acquiring junction resources, such as:

$R1 \rightarrow R2 \rightarrow R3 \rightarrow R4$

All processes must request resources in this order only. This eliminates circular wait and prevents future deadlocks.

Question 10 – University Examination Server Resource Allocation

1. Application of Banker's Algorithm

Banker's Algorithm is used to avoid deadlocks by checking every resource request before allocation.

When a student session requests CPU, memory, or database connections, the operating system first checks whether granting the request keeps the system in a safe state. If the system remains safe, the resources are allocated; otherwise, the request is postponed until resources become available.

2. Information Required Before Granting a Request

Before allocating resources, the operating system requires:

- Available Resources – Resources currently available in the system.
- Maximum Need – Maximum resources each student session may require.
- Allocated Resources – Resources already assigned to each session.
- Remaining Need – Resources still required to complete execution.

$\text{Remaining Need} = \text{Maximum Need} - \text{Allocated Resources}$

3. Determining a Safe State

The system checks whether there is a safe sequence in which all student sessions can complete without causing a deadlock.

If every session can finish one after another using the available resources, the system is considered to be in a safe state. Otherwise, the request is denied or delayed until the system becomes safe.

4. Limitation of Banker's Algorithm

A major limitation is that it requires the maximum resource requirement of every process to be known in advance. In large-scale cloud systems with thousands of dynamic user requests, predicting the maximum resource requirement is difficult. The algorithm also introduces additional computation, which can reduce performance in highly dynamic environments.